Review for Midterm of CS 135, Spring 2026

Contents

1 Rules for Taking an Exam **2**

2 Rules for Naming Variables **2**

3 Difference Between Variables and Constant Variables **3**
3.1 Need to Initialize a Primitive Type Variable before Using it . . . 3

4 Arithmetic Operators **3**
4.1 Division Operator . . . 3
4.2 Remainder or Modular Operator % . . . 3

5 Assignment Operator = **4**

6 Equality Opeator == **4**

7 Branch Statement **4**

8 Repetition Statements **5**
8.1 Syntax of Repetition Statement . . . 5
8.2 Difference Between while-statement / for-statement and do-while statement . . . 5
8.3 Usage of do-while statement . . . 5

9 Define Functions **6**
9.1 return statement vs. break statement . . . 6
9.2 Define a Function to Test Whether a Number is Prime or Not . . . 7
9.2.1 Test Whether 25 is a Prime or Not . . . 8
9.2.2 Test Whether 29 is a Prime or Not . . . 9
9.3 Print is Not the Same as Return . . . 9

10 Array **10**

11 Run Methods Associated with Objects of a Class **10**

12 File Input / Output (NOT Tested in The Midterm) **10**

13 Topics Not Tested In the Midterm **10**

14 Midterm Policy: Coding Shortcuts & Requirements **10**

15 Old Midterms **10**

1 Rules for Taking an Exam

1. The exam will be in paper. You must take the midterm in class room unless you got approval to take the exam in a different classroom in campus.

2. If you need accommodation, please take the exam at accessibility office. Our classroom will be reserved for other classes after the class ends. As a result, I cannot give you extra time or quiet environment.

3. Closed book, closed notes. No Internet or computer or cell phone or other electronic devices.

4. A review sheet at `https://maryash.github.io/135/slides/CheatSheet.pdf` will be provided (courtesy Professor Maryash). You cannot use your own review sheets.

5. No cheating. Work independently.

6. During the exam, you must place electronic devices, books, and notes in your bag and zip it closed.

   Any electronic device – including but not limited to calculators, AirPods, phones, smart glasses, or any type of wristwatch (analog, digital, or smart) – found outside your bag will be considered cheating and will result in a grade of zero. A timer will be displayed on the screen for the duration of the exam.

7. Sit in alternate rows and columns. That is, no two students can sit adjacent to each other.

8. Cannot go to bathroom when taking the exam.

2 Rules for Naming Variables

Rules for naming variables (including function or method names) in C++.

1. Start with letters or underscore symbol _

   Normally in variables in user application do not start names with _. Only variables in system coding will start with _.

2. Followed by letters, _, or digits.

3. Do not use keywords in C++ like if, int, while, and so on.

4. By convention, a constant variable is named with all capital letters. For example, to declare a variable representing number of hours per day, since this value is fixed, we use the following:

   ```
   const int HOURS_PER_DAY = 24;
   ```

3 Difference Between Variables and Constant Variables

Any variable is declared first before being used. Examples of usage include print out or put in right hand side expression to initialize or update a variable.

The difference between a variable and a constant variable is, the value of a variable can be updated from time to time. It is a like a cup that can hold different amount of water (values) at different time.

By contrast, a constant variable can only be initialized once. It is like a dumb bell, once it is created, its value is fixed and cannot be changed.

3.1 Need to Initialize a Primitive Type Variable before Using it

```
int i;
//i is an int and is NOT initialized yet

i += 5; //WRONG. i is not initialized yet
    .
```

```
std::string str;
//str is a string and is initialized
//to be an empty string, using
//the default constructor of string.

str += "Hello"; //ok
```

4 Arithmetic Operators

Arithmetic operators are +, -, *, /, and % (remainder or modular). They follow the same rules of combination and precedence in mathematics.

4.1 Division Operator

Pay special attention to division operator /.

When both numerator and denominator are integers, the quotient is also an integer. For example, expression 3 / 5 returns 0. This is similar to divide 3 pens among 5 students, and the number of pens each student gets is zero.

When at least one of numerator or denominator contains decimals, the quotient can keep its decimal parts. For example,

3.0 / 5 returns 0.6.

1.0 * 3 / 5 returns 3.0 / 5 = 0.6.

(double) 3 / 5 forces 3 to change to double type. Here (double) means type cast, which forces the immediately followed variable or value – in this example, number 3 – to become a double type, which is used to store values with decimal parts.

(double) (3 / 5) forces 3 / 5 into a double type. Since 3 / 5 returns 0 (integer division), so (double) (3 / 5) still returns 0.

1.0 * (3 / 5) returns 1.0 * 0 = 0.

4.2 Remainder or Modular Operator %

Result of 3 % 5 is 3, which can be interpreted as dividing 3 pens among 5 students, none of the students get any pen, and the number of pens left is 3.

Remainder operator is useful in "circle back" operation. For example, suppose now is 6 AM, what time was 8 hours AGO? Since number of hours we represent is 12, then (6 + 12 – 8) % 12 = 10. Similarly, what is 20 hours AFTER 6 AM? Then (20 + 6) % 12 = 2.

What is 20 hours BEFORE 6 AM, then it is (6 – 20 + 24) % 12 = 10. Where 24 is a multiple of 12.

This % operator is useful in Lab 6 string encrypt and decrypt.

5 Assignment Operator =

Assignment operator = runs from right to left. Evaluate the right hand expression first, then use its value to update the left hand variable. For example,

```
int a = 5;
a += 2; //the same as a = a + 2;
```

After the above segment, a is changed to 7.

6 Equality Opeator ==

Do not confuse assignment operator = with equality operator ==. In assignment operator, the right-hand side expression is evaluated, then use that value to set left-hand side variable, afterwards, the whole expression has that value. Equality operator compares the left-hand side expression with the right-hand side expression. If both sides equal, then return true (1 in C++), otherwise, returns false (0 in C++).

```
int a = 5;
cout << (a += 2) << endl; //a += 2 returns 7, print 7

cout << (a == 7) << endl; //print 1
cout << (a == 5) << endl; //print 0
```

7 Branch Statement

Branch statement includes if-, if-else and nested if-else statement. Nested if-else statements are mutually exclusive. For example, given an int, it can be positive, negative, or zero. Another example is traffic light and letter grades.

If conditions are not mutually exclusive, for example, grades and majors are not mutually exclusive, use separated if- statement.

For example, the following logic is wrong, it will only check major is in CS or not when grade is not 90. However, one can be in CS major when his/her grade is at least 90.

```
if (grade >= 90) {
   std::cout << "excellent\n";
}
else if (major == "CS") {
        std::cout << "Major is in CS\n";
}
```

8 Repetition Statements

For repetition statement, pay attention to initialization, condition to stop, what to repeat in each loop, how to update loop variable.

Some examples are in Labs 2, 3, 4, and class exercises on repetition statements.

8.1 Syntax of Repetition Statement

syntax of while-statement

```
initialize loop variable
while (condition) {
    things-to-do-in-one-round (including update loop variable)
}
```

syntax of for-statement

```
for (initialize loop variable; condition; update loop variable) {
    things-to-do-in-one-round
}
```

syntax of do-while statement

```
do {
    things-to-do-in-one-round
} while (condition); //do not forget ; to end do-while statement
```

8.2 Difference Between while-statement / for-statement and do-while statement

while-statement and for-statement run entry-check. That is, check the condition is satisfied or not before entering the loop.

for-statement put initialization of loop variable and update loop variable explicitly into for-header. In for-statement, loop variable is updated after things-to-do-in-one-round. For-statement is often used in situation when the number of repetitions is known in advance. do-while statement checks the condition at exit point. Statements in loop body of do-while statement runs at least once.

do-while statement can be rewritten as a while-statement.

```
thing-to-do-in-one-round //run loop body once
while (condition) {
    thing-to-do-in-one-round
}
```

8.3 Usage of do-while statement

do-while statement is used in situation when loop-body statement runs at least once. For example, find out number of digits of an integer by throwing away a digit a time and each integer has at least one digit.

The following code define a method to find out numbers of digits of an int.

```cpp
#include <iostream>
#include <cassert> //assert
using namespace std;

int getNumDigits(int n); //return number of digits of n
int main() {
    assert(getNumDigits(5) == 1);
    assert(getNumDigits(-5) == 1);
    assert(getNumDigits(0) == 1);
    assert(getNumDigits(69) == 2);
    assert(getNumDigits(-20) == 2);
    return 0;
}

int getNumDigits(int n) {
    int numDigits = 0;
    do {
        n /= 10; //throw the least significant digit from n
        numDigits++;
    } while (n != 0);

    return numDigits;
}
```

Another example for do-while statement is to enter an integer in [0, 100]. Since we need to take input at least once, we can use do-while statement.

9 Define Functions

When defining a function, remember to return in every possible branch and consider corner cases. For example, in the following isPrime function, need to consider when the input parameter is 1, 2, and 3 since for loop

for (int i = 2; i <= sqrt(num); i++) only works for num >= 4.

Reason: to enter the loop, we must have 2 <= sqrt(num). Then square both sides of the inequation, get num >= 4. As another example, to define a function to convert a numerical grade to a letter grade, consider the case when the input parameter is out of the range of [0, 100].

9.1 return statement vs. break statement

Return statement is very strong, it can be either in the form of `return;` if return type is void or return `a_value_of_non_void_return_type;`

Return statement leaves the current function and return to the caller function.

Compared with break statement, which can only be in the form of break;

Statement `break;` jumps out the current repetition block or switch block where break; statement resides.

9.2 Define a Function to Test Whether a Number is Prime or Not

```
#include <iostream>
#include <cmath>
using namespace std;

//if num is prime, returns true,
//otherwise, returns false.
bool isPrime(int num);

int main() {
    cout << isPrime(5);
    return 0;
}

//A positive int is a prime if and only if
//it can be divided by 1 AND itself only.
//By definition, is 1 a prime int? No.
bool isPrime(int num) {
    //for num, its trivial factors are 1 and num,
    //its POSSIBLE (candidates) of non-trivial factors are
    //2, ..., num/2.
    //We can shrink the range better.
    //Observation: non-trivial factors are paired.
    //For example, suppose num is 16,
    //then the non-trivial factors are 2, 4, 8
    //PAIRED non-trivial factors
    //2 * 8
    //4 * 4
    //8 * 2
    //The BIGGEST of smaller of PAIRED non-trivial factors is
    //sqrt(num).
    //Why? Suppose we have a pair of NON-trivial factors,
    //a and b, without loss of generality, we assume that a <= b.
    //a * b is num,
    //a <= b
    //so a is <= sqrt(num);

    //special processing for num equals to 1,
    //when num is 1, we do not enter the for loop,
    //without the following if-statement,
    //when num is 1,
    //we do not go through for-loop, then we return true.
    //That means 1 is prime, contradict to definition.
    if (num <= 1) { //negative integers are not prime, 1 is not prime.
       return false;
```

```
    }

    //for (int i = 2; i <= num/2; i++)
    //Why do we use sqrt(num) instead of num/2?
    //Suppose that num is 101, a prime num,
    //in the approach of
    //for (int i = 2; i <= num/2; i++)
    //    ...
    //we need to test every one of 2, 3, ..., 50.
    //But if we use
    //for (int i = 2; i <= sqrt(num); i++)
    //    ...
    //then we only need to test 2, 3, ..., 10 to find out that
    //101 is a prime.
    for (int i = 2; i <= sqrt(num); i++) {
        if (num % i == 0) { //i is an ACTUAL factor of num
           return false;
        }
    }

    return true; //return true ONLY after testing every non-trivial
    //factors of num and none of them becomes an ACTUAL factor of num.
    //like to test a batch of eggs.
    //for (every egg)
    //    if this current egg is bad
    //       return "bad quality"; //one bad means the whole batch is bad
    //
    //return "good quality" //good quality for every egg
}
```

9.2.1 Test Whether 25 is a Prime or Not

Here is a running to test whether 25 is a prime or not. Since 25 is not 1, we move to the next statement.

```
for (int i = 2; i <= sqrt(num); i++) {
    if (num % i == 0) { //i is an ACTUAL factor of num
       return false;
    }
}

return true;
```

Since sqrt(25) is 5, to run the above code when num is 25, replace sqrt(num) by 5.

```
for (int i = 2; i <= sqrt(num)5; i++) {
    if (num % i == 0) //i is an ACTUAL factor of num
       return false;
```

```
}

return true;
```

i = 2 (initialization)			
loop variable	condition	for-body	update loop variable

i	i <= sqrt(25)?	`if (num % i == 0) {`
		`  return false;`
		`}`
			i++

2	yes	25 % 2 is not 0, if-body does not run	3
3	yes	25 % 3 is not 0, if-body does not run	4
4	yes	25 % 4 is not 0, if-body does not run	5
5	yes	25 % 5 is 0, if-body runs, return false	

9.2.2 Test Whether 29 is a Prime or Not

Similarly, to test whether 29 is a prime or not, we go through the following segment. Note that sqrt(29) is 5.xxx, a decimal number between 5 and 6. So the maximum integer i satisfying i <= sqrt(29) is 5.

```
for (int i = 2; i <= sqrt(29); i++) {
    if (num % i == 0) { //i is an ACTUAL factor of num
       return false;
    }
}

return true;
```

i = 2 (initialization)			
loop variable	condition	for-body	update loop variable

i	i <= sqrt(29)?	`if (num % i == 0) {`
		`  return false;`
		`}`
			i++

2	yes	29 % 2 is not 0, if-body does not run	3
3	yes	29 % 3 is not 0, if-body does not run	4
4	yes	29 % 4 is not 0, if-body does not run	5
5	yes	29 % 5 is not 0, if-body does not run	6
6	no		

After moving out the loop, we run through its next statement

```
return true;
```

This means 29 is a prime.

9.3 Print is Not the Same as Return

Also, do not use cout to print results to screen unless the function concentrates on printing. Reason: the screen is a unique resource, if every function prints to the screen without coordination from main function, the screen will be a mess.

10 Array

An array is a group of same-type elements residing in consecutive spaces. The name of an array also implies the address of its first element. The index of an array always starts from 0. The last index of an array is one less than its size.

11 Run Methods Associated with Objects of a Class

Run operations for objects of a class. For example, given a string, how to find out the number of letters. For more exercises, see Lab 6.

12 File Input / Output (NOT Tested in The Midterm)

Also pay attention to file redirection in command line. For exercises, see Lab 3, Project 1, and E 8.1.

13 Topics Not Tested In the Midterm

Use-defined class and object will not be tested in the midterm. However, existing class like string will be tested.

Read from or write to a file (not tested in the midterm).

14 Midterm Policy: Coding Shortcuts & Requirements

Due to the limited time for this midterm, unless the problem explicitly specifies, you are encouraged to skip boilerplate code to focus on logic.

- **Standard Headers & Namespaces:** You do not need to include standard libraries or the using namespace std; directive.
- **Assumptions:** The environment assumes namespace std is active and all common headers (e.g., `<iostream>`, `<string>`) are already included.
- **Minimalist Requirements:** If a question asks you to define a function, provide only the function definition. You do not need to write a main() function or any setup code.

Example of what you can OMIT:

```
#include <iostream>
#include <string>
using namespace std;
```

15 Old Midterms

Old midterms and their answers are posted in webpage `https://tong-yee.github.io/135/old_midterms.html`.